

INFORME TÉCNICO

Laboratorio de rutas y algoritmos de búsqueda

Programación de Inteligencia Artificial · RA1

Nombre	Ignacio Pérez Imaz
Curso / grupo	Especialización de IA y Big Data
Fecha	01/10/2026

Introducción

Este informe explica el proceso seguido para construir una aplicación web de búsqueda de rutas sobre un tablero de 20×20. La aplicación permite editar el mapa, colocar inicio, metas, obstáculos y pesos, ejecutar BFS, DFS, Coste Uniforme y A*, y comparar sus resultados sobre el mismo tablero.

Estado actual del proyecto

La versión actual separa la lógica principal en script.js y la generación de escenarios en escenarios.js. La tabla comparativa empieza a cero y solo se calcula al pulsar “Comparar todos”. Si se modifica el tablero, los resultados se invalidan y vuelven a cero.

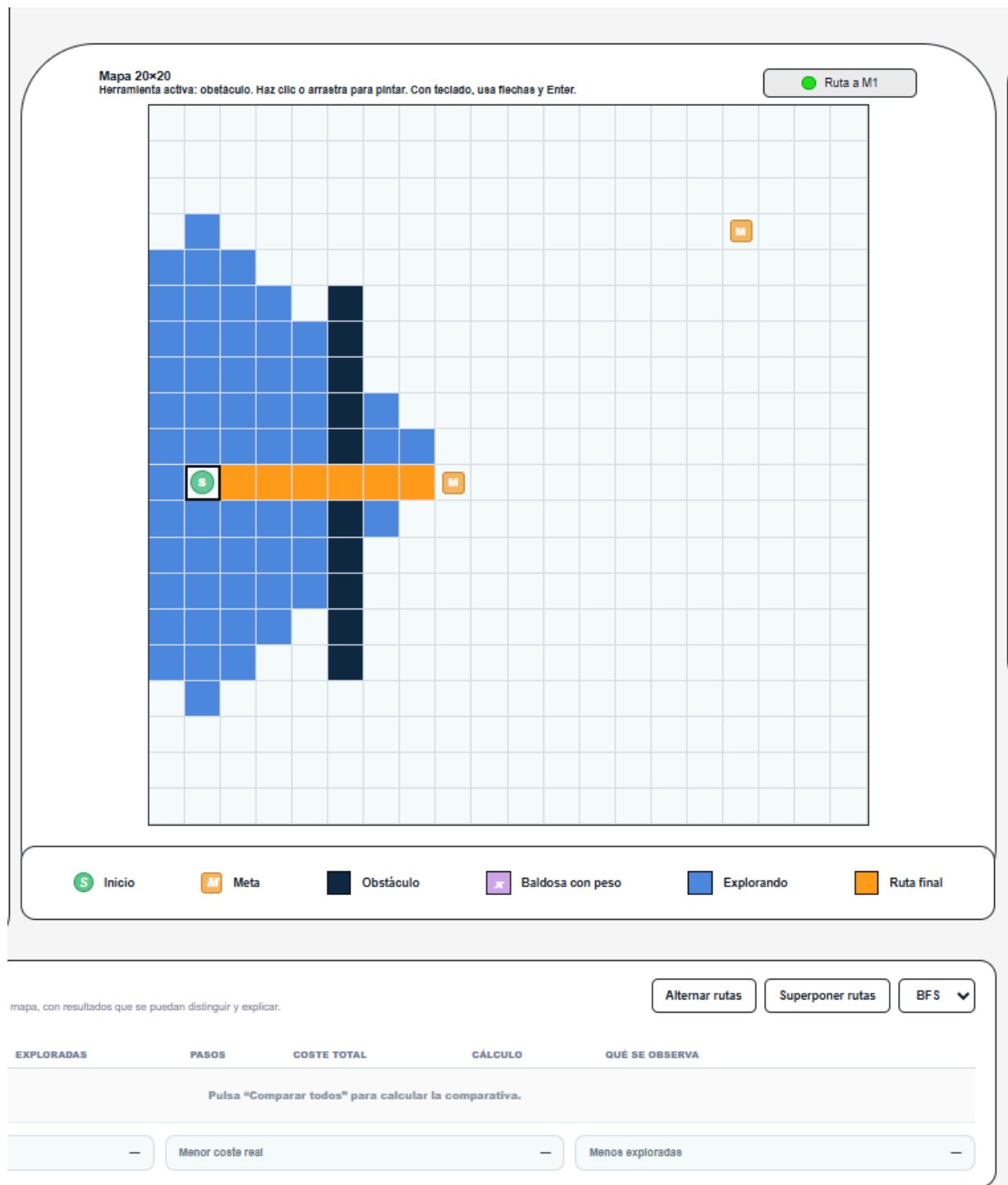


Figura 1. Vista general de la aplicación durante el desarrollo.

1. Paso a paso de construcción

Estructura inicial: Primero creé la página con HTML y dividí la interfaz en panel de controles, tablero, métricas y zona de comparación.

Tablero 20×20: Después generé una matriz de 20 filas y 20 columnas. Cada celda guarda si es muro y su coste, y además tiene una referencia al elemento visual que aparece en pantalla.

Herramientas de edición: Añadí las herramientas para pintar obstáculos, borrar, mover el inicio, colocar metas y asignar pesos. También añadí varias metas y una penalización de coste para poder comparar distancia y coste.

Primeros algoritmos: Implementé BFS y DFS para comprobar la exploración del tablero y la reconstrucción de la ruta.

Costes y algoritmos informados por coste: Después añadí los pesos de las celdas y programé Coste Uniforme (UCS) y A*. A* utiliza distancia Manhattan como heurística.

Animación y métricas: Añadí la animación de los nodos explorados, la ruta final y las métricas: meta, exploradas, pasos, coste de ruta, coste de meta y coste total.

Escenarios de prueba: Creé cuatro botones de escenario: Menos pasos, Pesos, Meta con coste y Sin solución. En la versión actual cada botón genera un mapa del tipo seleccionado.

Tabla comparativa: Una dificultad importante fue conseguir que la tabla utilizara siempre los datos del tablero actual. Finalmente la dejé a cero por defecto y solo se rellena al pulsar “Comparar todos”.

Separación en dos JavaScript: Al modificar la generación de escenarios se rompió la comparación. Para evitar que una parte afectara a la otra, mantuve la lógica de algoritmos y comparación en script.js y pasé la generación de escenarios a escenarios.js.

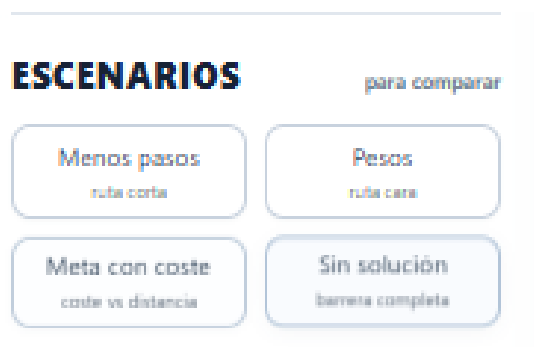


Figura 2. Botones utilizados para generar los cuatro tipos de escenario.

Capturas de los cuatro escenarios

Para la entrega final conviene sustituir los recuadros siguientes por capturas del tablero actual después de generar cada escenario.

Mapa 20x20

Herramienta activa: obstáculo. Haz clic o arrastra para pintar. Con teclado, usa flechas y Enter.

Ruta a M1

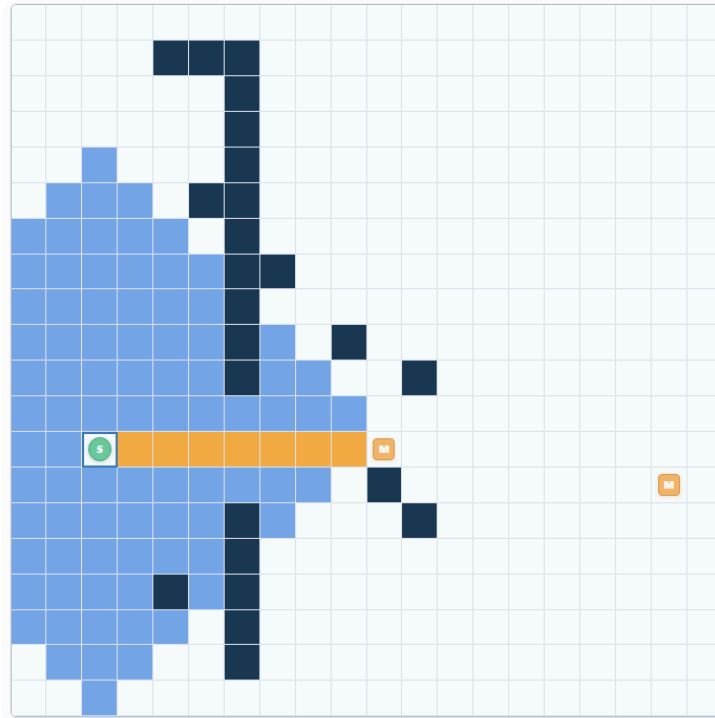


Figura 3. Espacio para la captura final del escenario “Menos pasos”.

Mapa 20x20

Herramienta activa: obstáculo. Haz clic o arrastra para pintar. Con teclado, usa flechas y Enter.

Ruta a M1

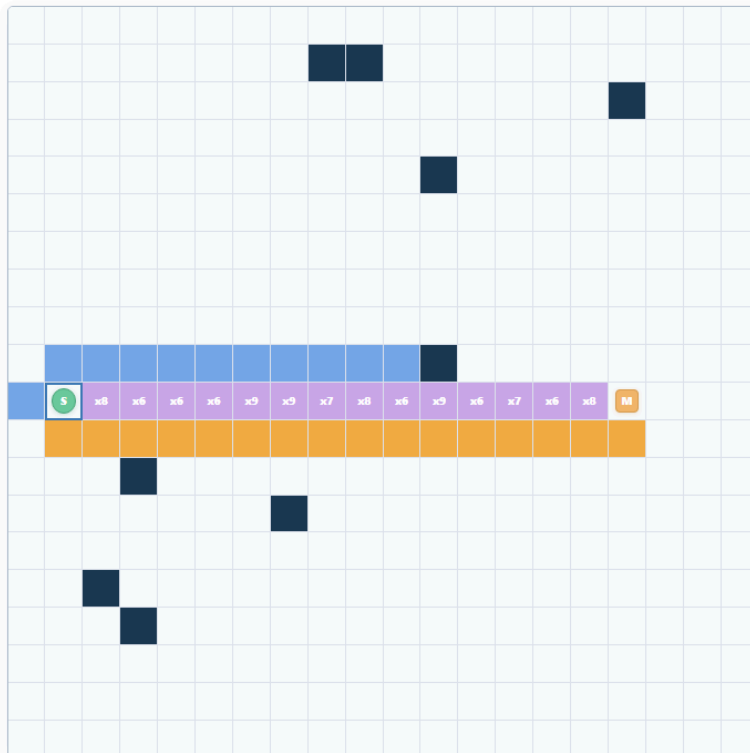


Figura 4. Espacio para la captura final del escenario “Pesos”.

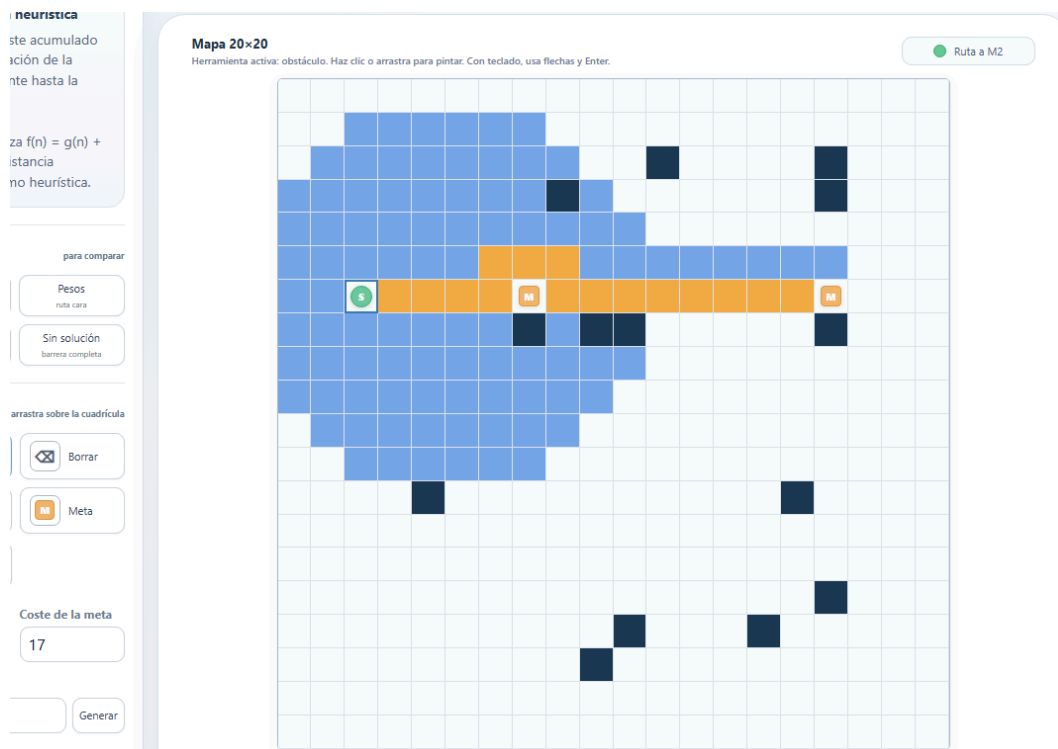


Figura 5. Espacio para la captura final del escenario “Meta con coste”.

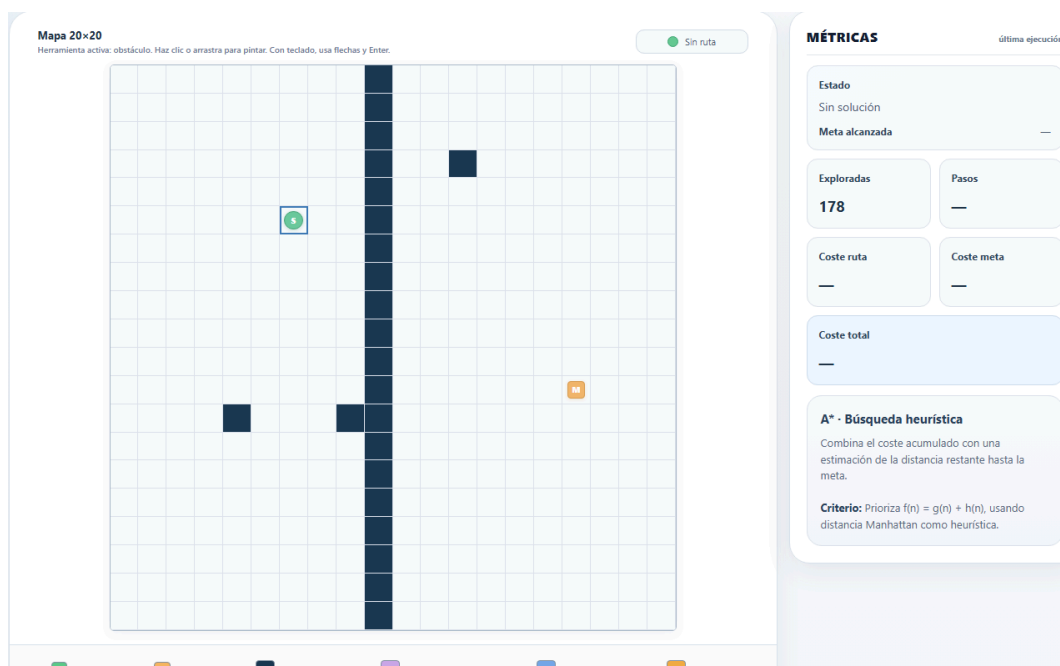


Figura 6. Espacio para la captura final del escenario “Sin solución”.

2. Arquitectura del proyecto

La aplicación está separada en archivos con responsabilidades diferentes:

Archivo	Función principal
index.html	Estructura de la interfaz y controles.
styles.css	Diseño, distribución, colores y estados visuales.

script.js	Tablero, herramientas, BFS, DFS, UCS, A*, animación, métricas y tabla comparativa.
scenarios.js	Generación de los cuatro tipos de escenario.
README.md	Documentación breve del proyecto.

Estructura de datos del mapa. El tablero se almacena en una matriz board de 20×20. Cada celda guarda principalmente wall, cost y la referencia visual el. El inicio se guarda en start y las metas en goals, donde cada meta incluye posición, penalización e identificador.

Flujo desde el clic hasta el resultado:

1. El usuario selecciona una herramienta o escenario.
2. El evento modifica board, start o goals.
3. Al pulsar Ejecutar se llama al algoritmo seleccionado.
4. El algoritmo devuelve nodos explorados, ruta y meta.
5. La interfaz anima la exploración y pinta la ruta final.
6. Las métricas se actualizan con los datos de esa ejecución.
7. Al pulsar Comparar todos se ejecutan los cuatro algoritmos sobre el mismo estado del tablero.

3. Explicación de BFS, DFS, UCS y A*

BFS

Usa una cola (FIFO) y explora el tablero por niveles. Marca las posiciones visitadas para no repetirlas. Cuando todos los movimientos cuestan lo mismo, encuentra una ruta con el menor número de pasos. No usa los pesos para decidir.

DFS

Usa una pila (LIFO) y profundiza por una rama antes de retroceder. También controla los visitados, pero no garantiza ni la ruta más corta ni la de menor coste.

UCS

Selecciona siempre el nodo pendiente con menor coste acumulado. Tiene en cuenta los pesos de las celdas y la penalización de la meta. En el proyecto la cola de prioridad se implementa ordenando un array.

A*

Combina el coste acumulado $g(n)$ con una heurística $h(n)$. La heurística utilizada es la distancia Manhattan, adecuada porque solo hay movimientos ortogonales. Su objetivo es dirigir mejor la búsqueda y reducir exploraciones innecesarias.

Visitados y ruta. Los algoritmos guardan qué posiciones ya se han procesado y una relación de padres para poder recuperar la ruta cuando se alcanza una meta.

Límites principales. El mapa está limitado a 20×20, solo permite movimientos ortogonales y A* utiliza únicamente distancia Manhattan. Para este tamaño, ordenar arrays en UCS y A* es suficiente, aunque en problemas grandes sería mejor utilizar un heap.

Binary Search. No se desarrolla en esta versión del informe porque en el enunciado se indica que sus detalles se explicarán más adelante.

4. Comparación de los cuatro laberintos

La versión actual genera un escenario nuevo cada vez que se pulsa uno de los cuatro botones. Por eso los valores exactos de pasos, coste y exploradas pueden cambiar entre ejecuciones. Para la

entrega final hay que elegir una ejecución de cada escenario, pulsar “Comparar todos” y copiar los valores obtenidos en las tablas siguientes.

COMPARATIVA
 Datos calculados en tiempo real con el mapa actual. Si cambias inicio, meta, obstáculos o pesos, la tabla se recalcula.

ALGORITMO	META	EXPLORADAS	PASOS	COSTE RUTA	COSTE META	COSTE TOTAL	CÁLCULO	QUÉ SE OBSERVA
BFS	M1	64	7	7	0	7	0.10 ms	BFS encontró 7 pasos. Para decidir ignora pesos y coste de meta; después medimos el coste real de su ruta (7).
DFS	M2	103	34	34	0	34	0.10 ms	DFS encontró una ruta de 34 pasos. No garantiza ni menos pasos ni menor coste; su coste real es 34.
Coste Uniforme	M1	64	7	7	0	7	0.20 ms	UCS si usa pesos y coste de meta. La ruta elegida tiene coste total 7.
A*	M1	8	7	7	0	7	0.00 ms	A* usa coste + heurística Manhattan. La ruta elegida cuesta 7 y exploró 8 nodos.

Figura 7. Ejemplo de la tabla comparativa después de calcular los cuatro algoritmos.

Escenario 1: Menos pasos

Debe mostrar la diferencia entre buscar pocos pasos y seguir una rama larga. BFS debería destacar en número de pasos y A* puede explorar menos nodos.

Algoritmo	Pasos	Coste total	Exploradas	Conclusión breve
BFS	10	10	124	Completar con la ejecución final
DFS	39	39	76	Completar con la ejecución final
UCS	10	10	124	Completar con la ejecución final
A*	10	10	27	Completar con la ejecución final

Escenario 2: Pesos

Debe incluir un camino corto caro y otro algo más largo barato. UCS y A* deberían preferir menor coste, mientras que BFS puede priorizar menos pasos.

Algoritmo	Pasos	Coste total	Exploradas	Conclusión breve
BFS	17	117	263	Completar con la ejecución final
DFS	37	37	58	Completar con la ejecución final
UCS	19	19	292	Completar con la ejecución final
A*	19	49	24	Completar con la ejecución final

Escenario 3: Meta con coste

Debe haber una meta cercana penalizada y otra más lejana barata. UCS y A* deberían decidir usando coste total.

Algoritmo	Pasos	Coste total	Exploradas	Conclusión breve
BFS	5	23	45	Completar con la ejecución final
DFS	46	46	95	Completar con la ejecución final
UCS	16	16	254	Completar con la ejecución final
A*	16	16	121	Completar con la ejecución final

Escenario 4: Sin solución

Una barrera completa impide llegar a la meta. Los cuatro algoritmos deben terminar e indicar que no existe ruta.

Algoritmo	Pasos	Coste total	Exploradas	Conclusión breve
BFS	_____	_____	177	Completar con la ejecución final
DFS	_____	_____	177	Completar con la ejecución final
UCS	_____	_____	177	Completar con la ejecución final
A*	_____	_____	177	Completar con la ejecución final

Conclusión general esperada

BFS destaca cuando importa el número de pasos; DFS puede encontrar una solución sin garantizar optimalidad; UCS tiene sentido cuando hay pesos; y A* usa la heurística para intentar mantener un buen coste explorando menos nodos.

5. Lenguaje de etiquetado y lenguaje de programación

HTML

HTML es un lenguaje de marcado. En este proyecto define la estructura: selector de algoritmo, botones, herramientas de dibujo, tablero, métricas y tabla comparativa. HTML indica qué elementos existen, pero no contiene la lógica de búsqueda.

CSS

CSS controla la presentación: distribución de paneles, colores, tamaños, estados de celdas, estilos de botones y apariencia de la tabla. No ejecuta los algoritmos.

JavaScript

JavaScript es el lenguaje de programación de la aplicación. Gestiona el estado del mapa, responde a los clics, ejecuta los algoritmos, calcula costes, anima la búsqueda y actualiza las métricas y la comparación.

HTML	CSS	JavaScript
------	-----	------------

Estructura y marcado	Presentación y estilos	Lógica y comportamiento
----------------------	------------------------	-------------------------

6. Mejoras implementadas

No he implementado ninguna mejora adicional aparte de los requisitos principales del ejercicio.